

JARVIS

Local, voice-driven AI assistant — system design & program baseline

● DRAFT

VERSION 0.1 · BASELINE

PROJECT JARVIS	DOCUMENT Engineering Design	VERSION 0.1
AUTHOR Achintya	DATE 2026-07-05	STATUS Draft
CLASSIFICATION Internal / Personal	LIFECYCLE Living document	

REV Revision History

VERSION	DATE	AUTHOR	SUMMARY OF CHANGES
0.1	2026-07-05	Achintya	Initial document. Establishes overview, vision, current state, scope, roadmap, and risk register. Baseline for all future documentation.
next	YYYY-MM-DD	—	[Describe changes here]

Maintenance policy. Every material change to scope, architecture, or roadmap must add a row here and increment the version. Minor edits (typos, formatting) do not require a version bump but should be noted in commit history.

01 Project Overview

What JARVIS Is

JARVIS is a **local, privacy-first, voice-driven AI assistant** that runs entirely on the user's own machine. It captures speech, reasons with a locally hosted large language model, executes real actions on the host system, and responds in a synthesized voice — with no dependency on cloud AI services for its core reasoning loop.

The assistant is modeled, in character and ambition, on the fictional J.A.R.V.I.S. from the *Iron Man* films: a composed, competent, always-available presence that remembers context, anticipates needs, and acts on the user's behalf.

Why It Exists

Commercial voice assistants are cloud-dependent, send user data off-device, and are closed to deep customization. Modern LLM assistants are powerful but live behind a chat window and forget everything between sessions. JARVIS exists to combine the strengths of both while eliminating their weaknesses:

- **Local and private** — reasoning, memory, and speech all run on-device.
- **Persistent memory** — it remembers across sessions using the user's own knowledge base.
- **Agentic** — it performs real actions, not just conversation.
- **Ambient and voice-first** — it is meant to be spoken to, not typed at.

The Problem It Solves

"An assistant that remembers my world, can act on my machine, and talks to me naturally — without sending my life to a cloud provider."

Concretely, JARVIS addresses three gaps: **memory** (conversations and facts persist and are recalled), **agency** (it controls the desktop through a governed tool interface), and **presence** (a hands-free, wake-word voice loop rather than a chat box).

Intended Users

- **Primary** — the project author, as a daily-driver personal assistant and coding companion.
 - **Secondary** — developers and enthusiasts interested in local-first, agentic AI.
 - **Tertiary (aspirational)** — open-source users wanting a private, extensible assistant.
-

02 Vision

JARVIS aims to become a **genuine daily-driver AI companion** built on three pillars:

PILLAR	MEANING
Remembers your world	Projects, notes, goals, and past conversations — stored as inspectable, portable knowledge the user also owns.
Codes alongside you	Reads, writes, and runs code in a governed workspace, driven by voice.
Converses like the movie	Natural, low-latency, interruptible speech with a consistent persona and ambient proactivity.

The long-term north star is an assistant indistinguishable in *feel* from the fictional JARVIS: present without being intrusive, capable without being dangerous, and personal without being cloud-bound.

03 Objectives

FUNCTIONAL OBJECTIVES

ID	OBJECTIVE
F-1	Reliable hands-free voice interaction (wake word → command → spoken response).
F-2	Persistent, semantically searchable memory across sessions.
F-3	Governed execution of real-world actions (apps, files, system, notes, timers).
F-4	Proactive goal tracking and reminders.
F-5	Grounded factual answers via a curated knowledge corpus.
F-6	A consistent, movie-accurate conversational persona.

TECHNICAL OBJECTIVES

ID	OBJECTIVE
T-1	Fully local reasoning loop (no cloud LLM dependency).
T-2	Event-driven, loosely coupled architecture with sub-millisecond dispatch.
T-3	A hardened tool-execution surface safe against prompt-injection-driven misuse.
T-4	Reproducible builds, automated tests, and continuous integration.
T-5	Graceful degradation when subsystems (LLM, embeddings, microphone) are unavailable.
T-6	Latency targets: first audio < 1.5 s (tool routes), < 3 s (LLM routes). <i>Target; not yet benchmarked</i> — see §8.

04 Current Development Status

JARVIS is a **functional, end-to-end assistant** in active development. The reasoning, memory, action, and voice loops all work. The codebase is

approximately **8,000 lines** of application code plus a **127-case automated test suite**, under continuous integration. Development currently centers on the third-generation orchestrator (V3); a stable second-generation orchestrator (V2) remains runnable.

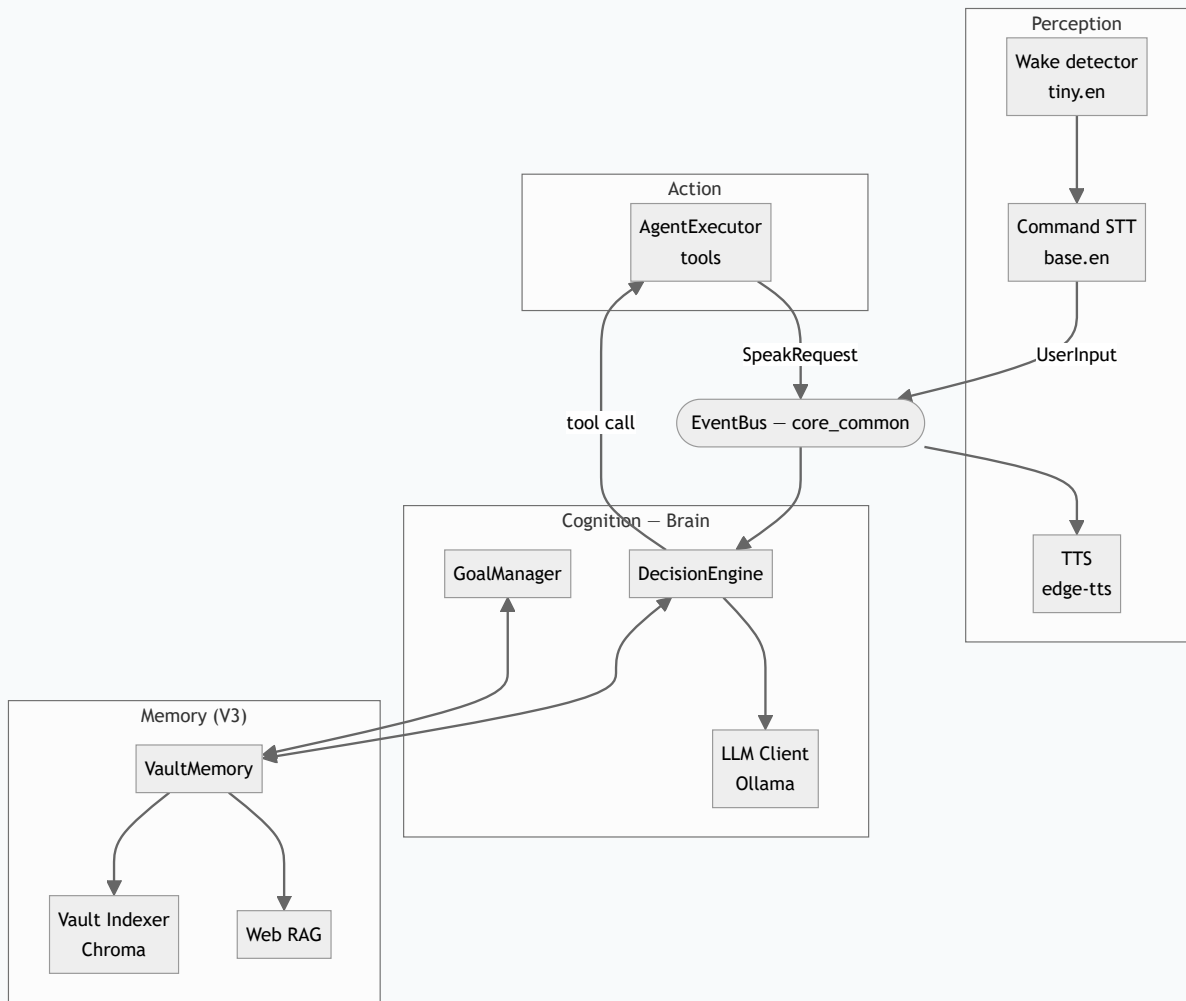
Technologies & Frameworks

CATEGORY	TECHNOLOGY
Language / Runtime	Python 3.10+
Local LLM	Ollama — qwen2.5:7b-instruct (fallback 3b)
Speech-to-Text	faster-whisper — base.en capture, tiny.en wake
Text-to-Speech	edge-tts (neural voices)
Embeddings	Ollama nomic-embed-text
Vector Store	ChromaDB
Persistent Memory	Obsidian markdown vault (V3); SQLite + ChromaDB (V2)
Audio I/O	sounddevice, miniaudio
Desktop Control	pyautogui, pygetwindow, pycaw
Testing / QA	pytest, pytest-asyncio, ruff, GitHub Actions CI
Packaging	pyproject.toml

Overall Architecture

JARVIS uses an **event-driven, single-process architecture**. An in-memory `EventBus` decouples subsystems; thin *adapters* bridge each subsystem to the bus. Shared primitives live in a neutral `core_common/` package used by all orchestrator generations.

FIG 10.A SYSTEM ARCHITECTURE



LAYERED STRUCTURE

LAYER	PACKAGE(S)	RESPONSIBILITY
Perception	<code>perception/</code>	Mic ownership, wake detection, STT, TTS, cues
Cognition	<code>cognition/</code>	Decision engine, agent state, LLM client, goals, agent-core loop
Action	<code>action/</code>	Fast intent router, tool executor, desktop/web/vault tools
Memory	<code>memory/</code>	Vault I/O, semantic indexer, web RAG, shared embedder (V3)
Orchestration	<code>core_v2/</code> , <code>core_v3/</code>	Event-driven Brain wiring the layers together
Shared	<code>core_common/</code> , <code>core/</code> , <code>infra/</code>	Event bus/types, config, resource monitor

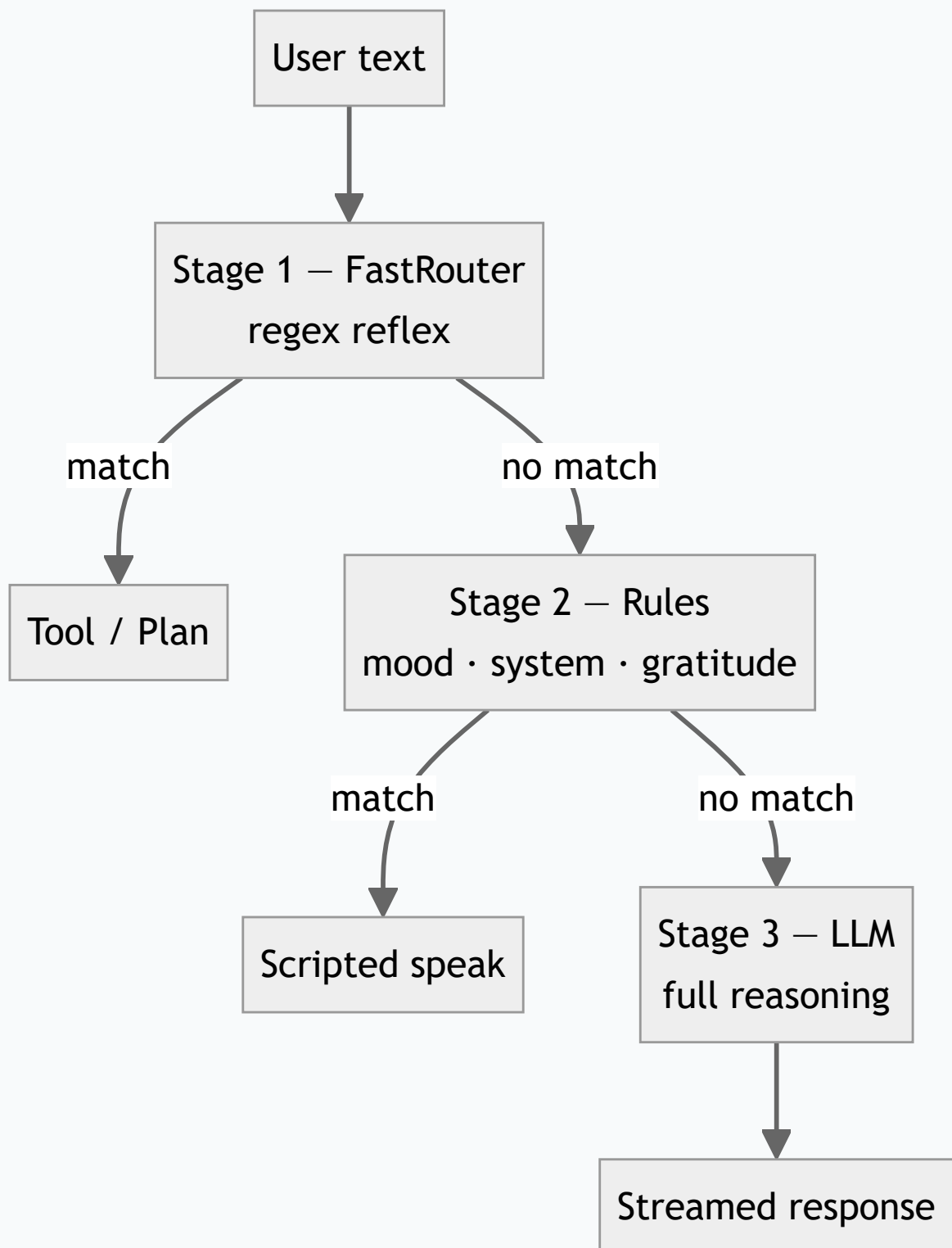
ORCHESTRATOR GENERATIONS

GEN	PACKAGE	STATUS	NOTES
V1	<code>legacy/</code>	RETIRED	Original monolith; reference only
V2	<code>core_v2/</code>	● STABLE	SQLite + Chroma memory · <code>python main.py</code>
V3	<code>core_v3/</code>	● ACTIVE	Vault memory, wake voice · <code>python -m core_v3</code>

Features Completed

AREA	COMPLETED CAPABILITY
Voice input	Single-mic wake-word loop; fuzzy wake matching; command capture
Voice output	Neural TTS with tunable cadence; queue-based streaming playback
Barge-in	Interrupt a spoken reply with the wake word (half-duplex)
STT robustness	Whisper hallucination gating (confidence + phantom-phrase filtering)
Reasoning	3-stage pipeline: FastRouter (reflex) → rules → local LLM
Memory (V3)	Obsidian vault: daily logs + semantic recall across notes
Goals	Voice-created goals persisted as Obsidian tasks; proactive reminders
Tools	Time, timers, system info, volume, clipboard, app/URL launch, files, shell, notes
Knowledge (code)	FineWeb web-knowledge RAG pipeline (index build is a user step)
Security	Allowlisted shell, sandboxed files, fail-closed confirmation
Engineering	Pinned deps, 127-test suite, CI, README, lint-clean, core_common

FIG 10.D DECISION PIPELINE



05 Work in Progress

FEATURES UNDER DEVELOPMENT

ITEM	STATUS	NOTES
Barge-in reliability tuning	IN PROGRESS	Improving hit-rate & latency on speaker setups
V3 ⇄ V2 cutover (Phase E)	PENDING	Retire V2 once V3 dogfoods cleanly
Coding partner (Phase G)	PLANNED	Voice-driven code read/edit/run in a governed workspace

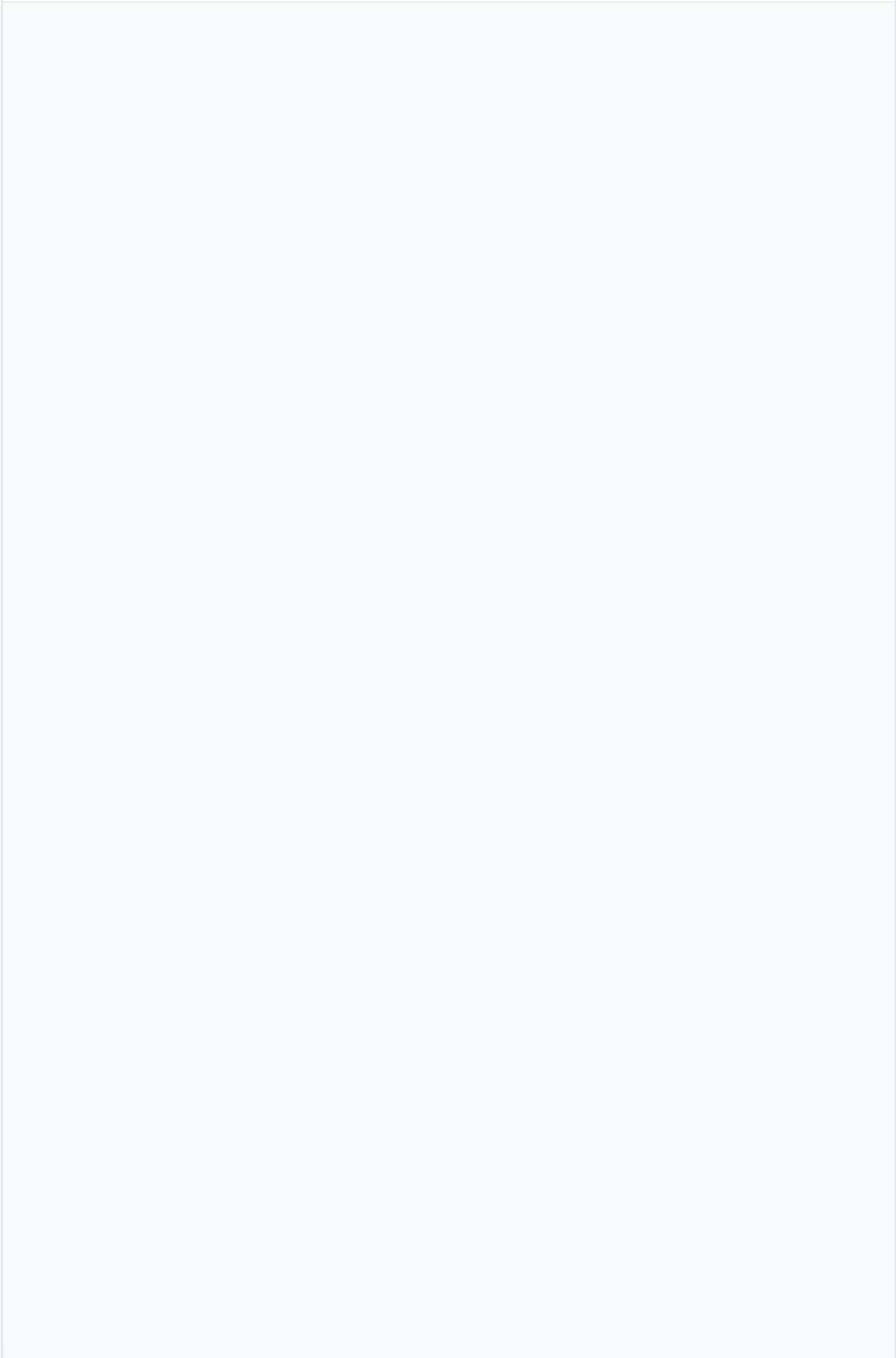
KNOWN LIMITATIONS

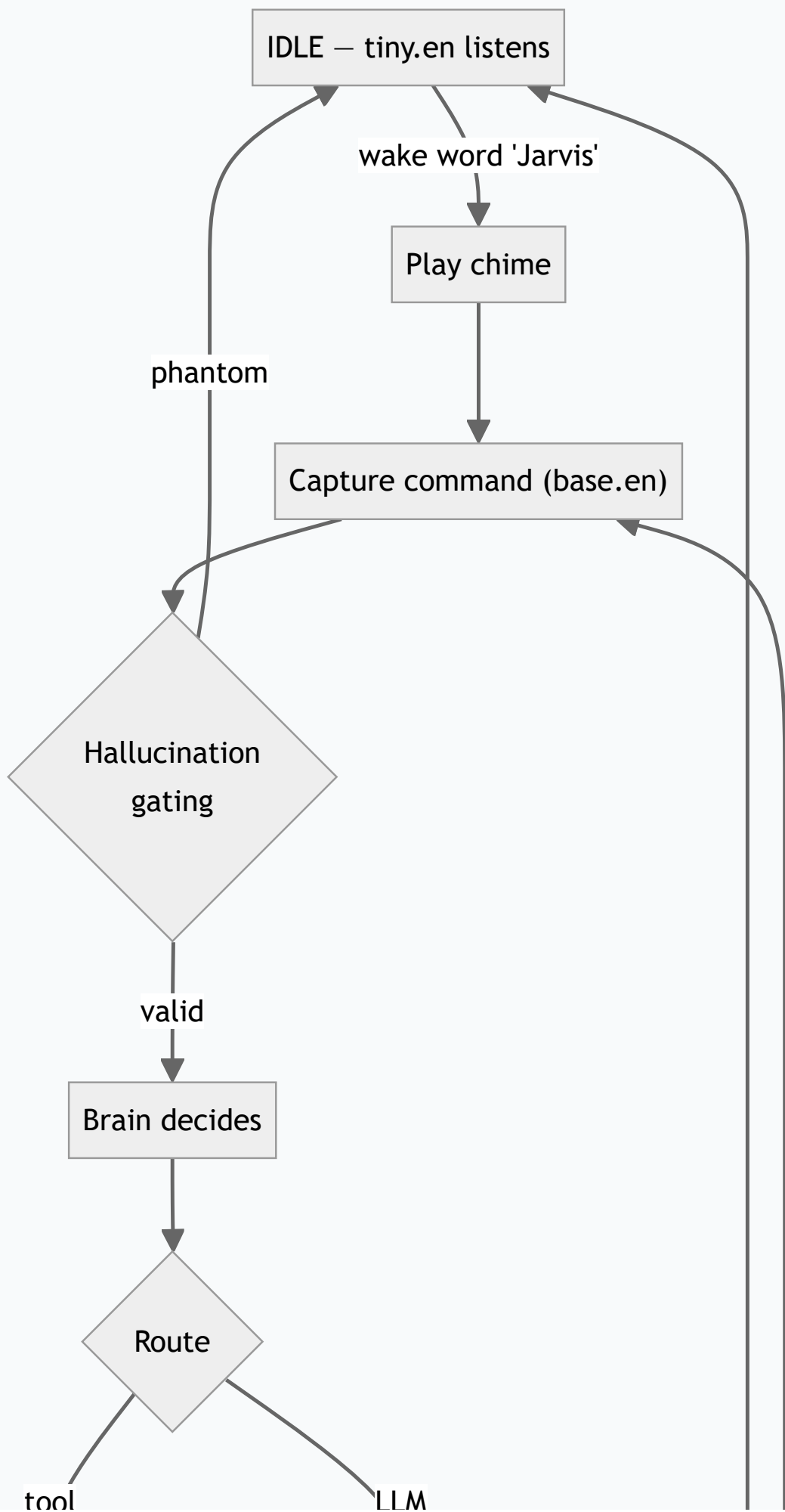
- Barge-in is best-effort on speakers; not yet 100% reliable without headphones or AEC.
- The web-knowledge corpus is implemented but not built/populated by default.
- Latency targets (T-6) are not yet formally measured.
- Windows-only for desktop-control features.

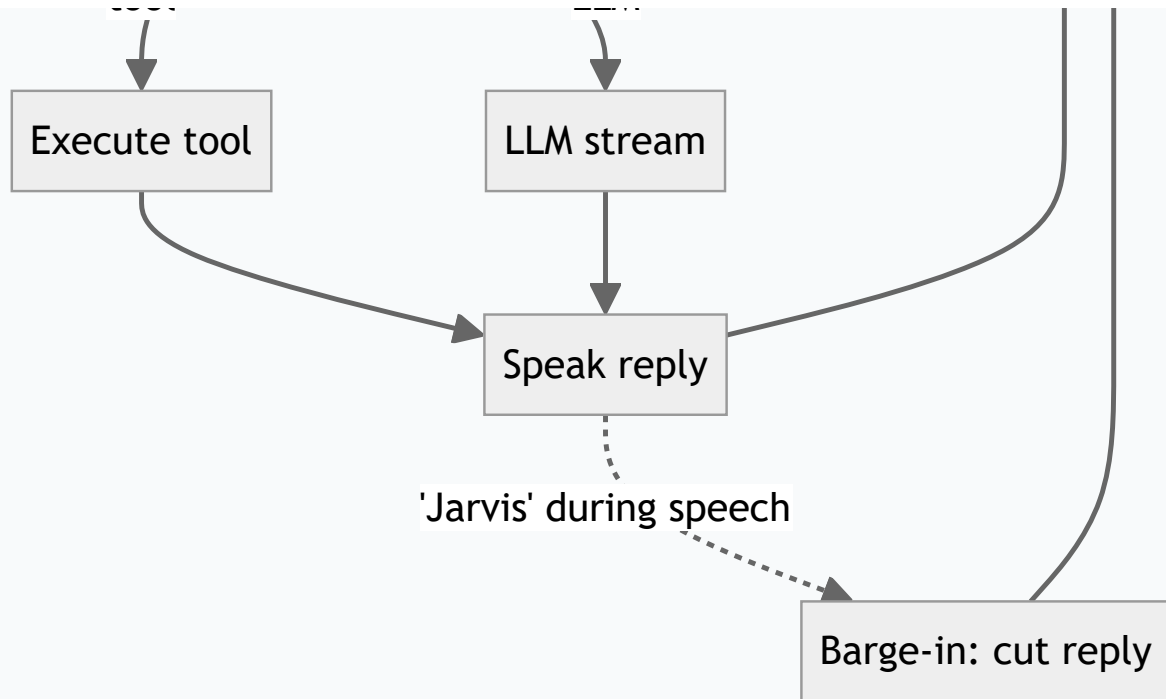
TECHNICAL BLOCKERS

ID	BLOCKER	IMPACT
B-1	No acoustic echo cancellation	Caps barge-in reliability on speakers
B-2	Host RAM pressure under full stack	Limits model size / concurrency headroom
B-3	[placeholder – record new blockers]	–

FIG 10.B VOICE INTERACTION FLOW







06 Scope

IN SCOPE	OUT OF SCOPE (CURRENT PHASE)
Local, single-machine, single-user operation	Cloud-hosted LLM inference
Hands-free wake-word voice interaction	Cross-platform desktop control (Windows-first)
Persistent, user-owned memory (Obsidian vault)	Multi-user / multi-device
Governed desktop / system / file / note actions	Distributed architecture / message-queue backbone
Local LLM reasoning + optional web-knowledge grounding	Model training or fine-tuning
A movie-accurate conversational persona	Full acoustic echo cancellation (deferred)
	Mobile or web front-ends

Items may move from Out of Scope to In Scope in future revisions; such moves must be recorded in the Revision History.

07 Roadmap

Horizon	Milestones
Immediate now	Push committed codebase to remote · finalize barge-in tuning · ratify this document as the baseline
Short-term next	Phase E — cutover, retire V2/V1 to legacy/ · Phase G — coding partner · formal latency benchmarking
Medium-term	Local voice cloning (XTTS) with edge-tts fallback · calendar & media integrations · build/enable web-knowledge corpus · benchmarks + demo
Long-term	Ambient always-available presence · safe plugin/tool ecosystem · release-grade reliability as an open reference · [placeholder]

08 Risks & Challenges

ID	RISK / CHALLENGE	LIKELIHOOD	IMPACT	MITIGATION / STATUS
R-1	Speaker echo caps barge-in reliability (no AEC)	High	Medium	Wake-gating + half-duplex + fuzzy matching; AEC deferred
R-2	Host resource pressure (RAM/VRAM)	Medium	Medium	Wake-gated STT; smaller fallback model; edge-triggered alerts
R-3	Local LLM quality ceiling vs. cloud	Medium	Medium	Model-agnostic client; upgrade path as local models improve
R-4	Multi-generation architecture debt	Medium	Low	core_common done; Phase E cutover planned
R-5	Prompt-injection-driven tool misuse	Low	High	Allowlisted shell, sandbox, fail-closed confirmation, tests
R-6	Single-machine data durability	Medium	High	Version control; user-owned vault; remote push
R-7	Latency exceeding conversational comfort	Medium	Medium	FastRouter reflex path; benchmarking pending

OPEN DESIGN UNCERTAINTIES

- Whether to invest in full acoustic echo cancellation vs. remaining wake-gated. *(Undecided.)*
- The build-vs-delegate decision for the Phase G coding capability. *(To be decided at Phase G.)*

09 Future Documentation

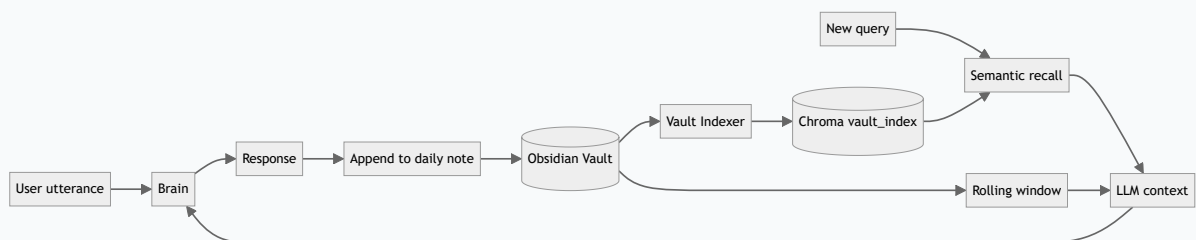
This document is the root of a planned documentation suite. The following should be authored as their subjects stabilize:

PLANNED DOCUMENT	PURPOSE	PRIORITY
Architecture Specification	Component design, event catalog, data flow	High
Voice & Perception System	Wake loop, STT/TTS pipeline, barge-in, tuning	High
Memory System	Vault schema, indexing, recall, retention	High
AI Models & Reasoning	Model selection, prompts, decision pipeline	Medium
Tool / Action Reference	Full tool catalog, arguments, permissions	Medium
Security Model	Threat model, sandboxing, confirmation policy	Medium
Testing Strategy	Test taxonomy, coverage goals, CI policy	Medium
Deployment & Setup Guide	Installation, prerequisites, configuration	Medium
Hardware Integration	Microphone / audio device requirements	Low
Plugin / Extension Guide	Adding new tools and integrations	Low
Benchmarks & Metrics	Latency, routing accuracy, STT accuracy	Low

10 Appendix

10.C – MEMORY DATA FLOW

FIG 10.C CONVERSATION → VAULT → INDEX → RECALL



10.E - DIRECTORY / MODULE MAP

```
jarvis_v1/
├─ core_common/      # Shared: EventBus, event types, SpeakingGuard
├─ core/              # Configuration
├─ infra/             # Resource monitor (CPU/RAM/VRAM pressure)
├─ perception/        # Wake detection, STT, TTS, audio cues
├─ cognition/         # Decision engine, agent state, LLM, goals, memory ctx
│   └─ agent_core/    # Multi-turn tool-use loop
├─ action/            # FastRouter, AgentExecutor, tools
├─ memory/            # (V3) Vault I/O, indexer, web RAG, embedder
├─ core_v2/           # V2 orchestrator (stable)
├─ core_v3/           # V3 orchestrator (active target)
├─ data/              # Offline FineWeb pipeline + blocklists
├─ tests/             # Automated test suite (pytest)
├─ tools/             # Developer utilities
├─ legacy/            # V1 orchestrator (reference only)
└─ docs/              # Engineering documentation (this document)
```

OTHER APPENDICES

REF	ITEM	STATUS
10.F	Screenshots / demonstrations	[placeholder]
10.H	References (repo, related docs, external tools)	[placeholder]

TERM	DEFINITION
Wake word	The spoken trigger ("Jarvis") that begins a command capture.
Barge-in	Interrupting an in-progress spoken reply by speaking.
Half-duplex	Listening and speaking are mutually exclusive to avoid self-hearing.
Vault	The user's Obsidian markdown knowledge base used as JARVIS's memory.
RAG	Retrieval-Augmented Generation — grounding answers in a retrieved corpus.
FastRouter	The zero-latency regex-based intent router (reflex stage).
Orchestrator	The Brain that wires perception, cognition, action, and memory.